

Final Project

AHB VIP

Group Members:
Usama Ahmed (Lead)
Ahmad Mukhtar
Khizer Mehmood

1 Verification Plan

No.	Test case	Description	Type	Status	Comment
1.	Single burst read after write test	Write to a specific address using a single burst and then read from the same address using single burst	Directed		
2.	INCR4 read after write burst test	First write to specific addresses using INCR4 burst and then read from those address using the same INCR4 burst. Also include busy states from the slave UVC to mimic real-life master behavior	Directed		

3.	INCR8 read after write burst test	First write to specific addresses using INCR8 write burst along with BUSY states from slave UVC after that read the data from those address to verify using the same INCR8 burst but in read mode	Directed		
4.	INCR16 read after write burst test	First write to specific addresses using INCR16 burst and then read from those address using the same INCR16 burst. Also include busy states from the slave UVC to mimic real-life master behavior	Directed		
5.	WRAP4 read after write burst test	Now this time use the WRAP4 write burst to write to a specific set of addresses using wrapping	Directed		

		around and then read from them to verify the data using the same WRAP4 burst			
6.	WRAP8 read after write burst test	Use the WRAP8 burst to write and then read sets of addresses and verify	Directed		
7.	WRAP16 read after write burst test	Use the WRAP16 write burst to write specific set of addresses using the wrapping logic and then read from those addresses to verify the data using the same WRAP16 read burst. The BUSY states from the slave UVC are inserted to mimic the behavior of master	Directed		
8.	Undefined length INCR read after write burst test	Use the undefined length INCR burst type using 4 beats to write to	Directed		

		certain addresses and then read from those addresses using the same undefined length INCR burst to verify the data read and also include BUSY states from master			
9.	Idle read after write test	Use the single write burst to write to a specific address while keeping the HTRANS == IDLE and then similarly use the single read burst to read from the same address keeping HTRANS == IDLE and verify that no read or write has happened	Directed		
10.	Waited transfer Idle-to-Non-seq test	During the waited transfer of a non-seq transaction change state	Directed		

		to idle and then from idle to non-seq and verify the address phase stays till HREADY == 1			
11.	INCR 1KB test	Do the INCR burst transaction of undefined length crossing the memory addresses of 1KB and verify the HRESP of the slave DUT	Directed		
12.	Locked transfer test	Do a burst sequence making a locked transfer and verify the locking and unlocking functionality of the slave	Directed		

Note: The response of the slave is mimicked in such a way that its outputs HREADYOUT and HRESP are generated in a random fashion and UVCs communicate in such a way that when HREADY is 1 based on HREADYOUT, HBUSY or HRESP then and only then the communication happens if for some case HREADY is 0 then both the master and slave UVC waits. Also, in case of HRESP == 1 the transactions are terminated by the driver with HTRANS == IDEAL and waits for the sequence to drop objection.

2 Assertions

No.	Assertion	Description	Status	Comment
1.	error_behaviour	Checks the error response of the slave DUT specifically checks the HRESP stays high for 2 cycles		Assertion on the Slave UVC side that checks outputs of the slave DUT
2.	error_happening_transaction_termination	Checks that on the First cycle of HRESP, HREADY is 0 and 1 in the 2 nd cycle		Assertion in the Slave UVC
3.	single_burst_read_after_write	Checks a single Burst read transaction followed by an ideal one then a single burst read transaction followed by ideal		Master UVC assertion
4.	INCR4_start	Verifies the starting conditions of INCR4 burst		-
5.	INCR4_addr_incr_on_HReady	Verifies that during an INCR4 burst address is incremented each time HREADY == 1		-
6.	INCR4_addr_incr_on_BUSY	Verifies if HREADY == 1		-

		followed by a HBUSY == 1 the previous address value is incremented		
7.	INCR4_control_signals_retention	Verifies that during the whole INCR4 burst the control signals remain same		-
8.	INCR8_start	Verifies that correct start of the INCR8 burst		-
9.	INCR8_addr_incr_on_HReady	Verifies that during the INCR8 burst address is incremented on each HREADY == 1		-
10.	INCR8_addr_incr_on_BUSY	Verifies that each time HREADY == 1 followed by a HBUSY == 1 the address is incremented		-
11.	INCR8_control_signals_retention	Verifies the retention of control signals for the complete INCR8 burst		-
12.	INCR16_start	Verifies that correct start of the INCR16 burst		-
13.	INCR16_addr_incr_on_HReady	Verifies that during the		-

		INCR16 burst address is incremented on each HREADY == 1		
14.	INCR16_addr_incr_on_BUSY	Verifies that each time HREADY == 1 followed by a HBUSY == 1 the address is incremented		-
15.	INCR16_control_signals_retention	Verifies the retention of control signals for the complete INCR16 burst		-
16.	WRAP4_start	Verifies that correct start of the WRAP4 burst		-
17.	WRAP4_wrap_around_check	Verifies that wrap around phenomenon of the WRAP type burst and makes sure that the address stays within the wrap boundary		-
18.	WRAP4_addr_incr_on_HReady	Verifies that during the WRAP4 burst address is incremented on each HREADY == 1 or wrapped around in case the		-

		boundary is reached		
19.	WRAP4_addr_incr_on_BUSY	Verifies that each time HREADY == 1 followed by a HBUSY == 1 the address is incremented or wrapped around if boundary is reached		-
20.	WRAP4_control_signals_retention	Verifies the retention of control signals for the complete WRAP4 burst		-
21.	WRAP8_start	Verifies that correct start of the WRAP8 burst		-
22.	WRAP8_wrap_around_check	Verifies that wrap around phenomenon of the WRAP type burst and makes sure that the address stays within the wrap boundary		-
23.	WRAP8_addr_incr_on_HReady	Verifies that during the WRAP8 burst address is incremented on each HREADY == 1 or wrapped around in		-

		case the boundary is reached		
24.	WRAP8_addr_incr_on_BUSY	Verifies that each time HREADY == 1 followed by a HBUSY == 1 the address is incremented or wrapped around if boundary is reached		-
25.	WRAP8_control_signals_retention	Verifies the retention of control signals for the complete WRAP8 burst		-
26.	WRAP16_start	Verifies that correct start of the WRAP16 burst		-
27.	WRAP16_wrap_around_check	Verifies that wrap around phenomenon of the WRAP type burst and makes sure that the address stays within the wrap boundary		-
28.	WRAP16_addr_incr_on_HReady	Verifies that during the WRAP16 burst address is incremented on each		-

		HREADY == 1 or wrapped around in case the boundary is reached		
29.	WRAP16_addr_incr_on_BUSY	Verifies that each time HREADY == 1 followed by a HBUSY == 1 the address is incremented or wrapped around if boundary is reached		-
30.	WRAP16_control_signals_retention	Verifies the retention of control signals for the complete WRAP16 burst		-
31.	undef_INCR_start	Verifies the correct start of the undefined length INCR sequence		-
32.	undef_INCR_addr_incr_on_HReady	Verifies the correct address update of undefined length INCR sequence based on the value of HREADY		-
33.	undef_INCR_addr_incr_on_BUSY	Verifies the correct updating of		-

		address when HREADY == 1 and is followed by HBUSY == 1		
34.	undef_INCR_control_signals_retention	Verifies the correct retention of the control signals during the undefined length INCR sequence		-
35.	locked_transfer_1	Verifies the correct termination of a locked sequence		-
36.	locked_transfer_2	Verifies that a transaction is locked and stays locked until its finally unlocked, thus verifying the overall functionality of a locked transfer		-
37.	idle_transfer	Verifies that during a waited transfer the state can change from idle to non- seq and will stay same until HREADY == 1		-